# Digital Image Processing

Dr. Muhammad Sajjad
R.A: Asad Ullah

Lecture 09

## Image Segmentation

# Overview

1 Fundamentals of Segmentation

2 Thresholding Techniques

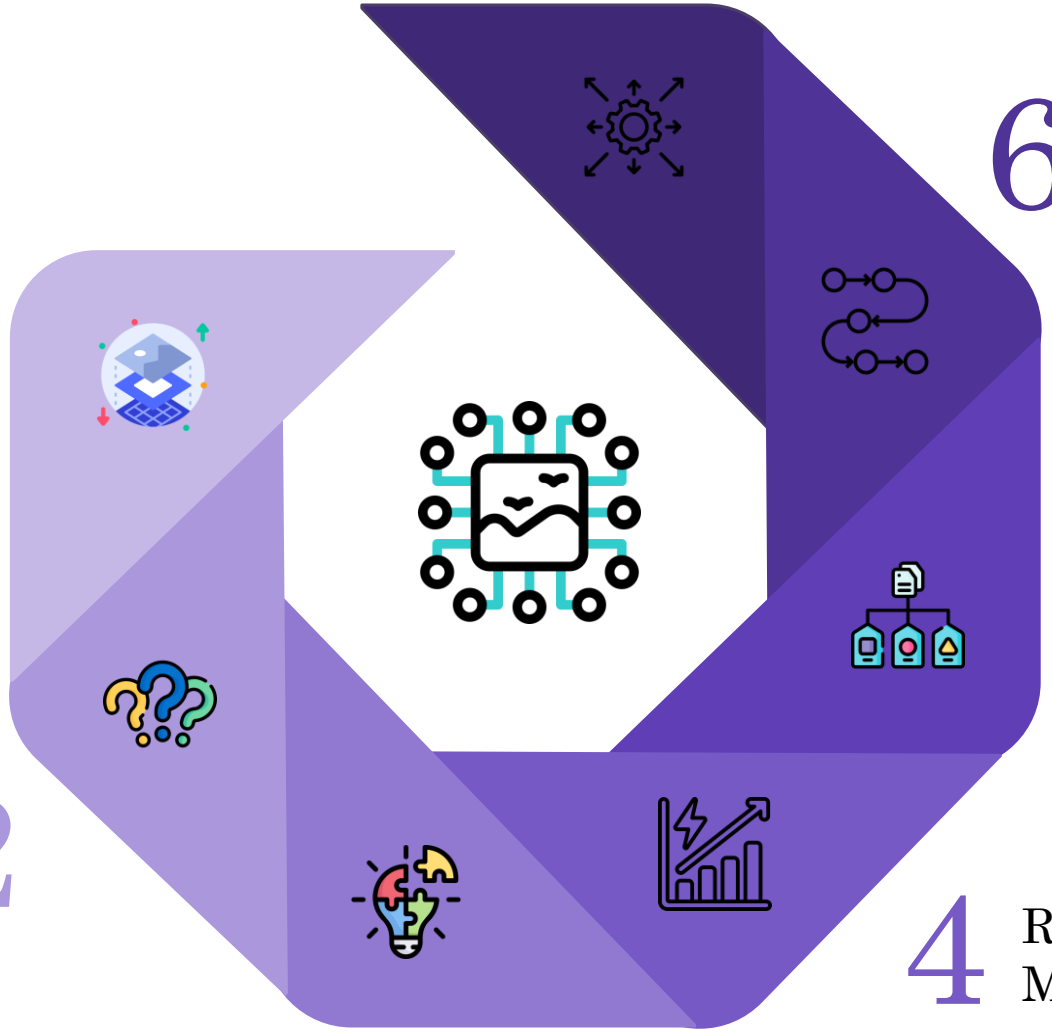
3 Edge-Based Methods

4 Region-Based Methods

5 Clustering Algorithms

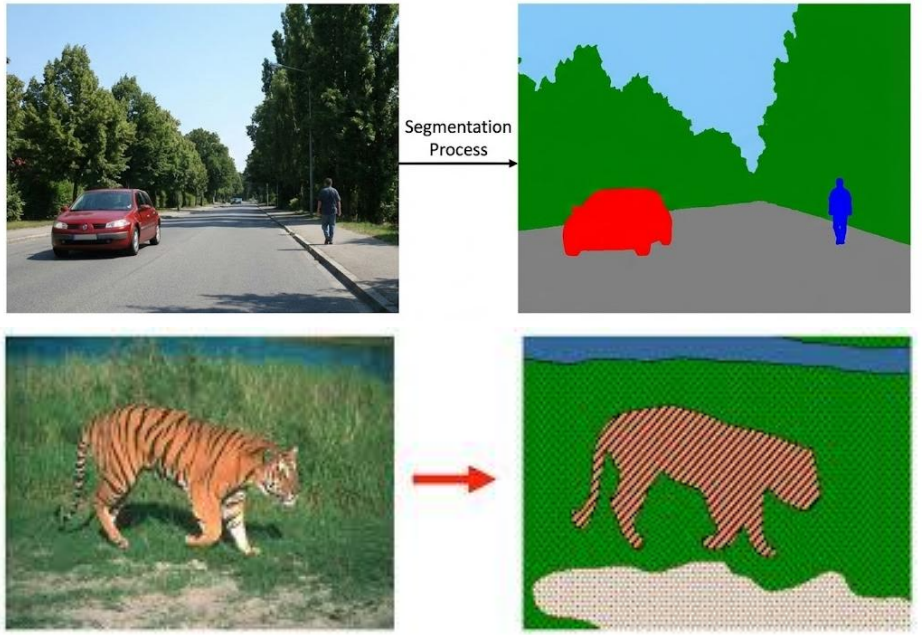
6 Graph-Based Segmentation

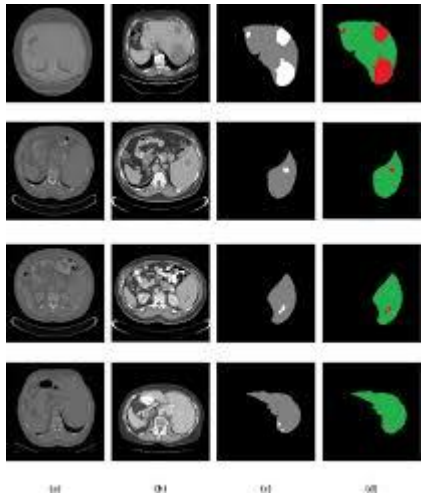
7 Advanced Classical Models

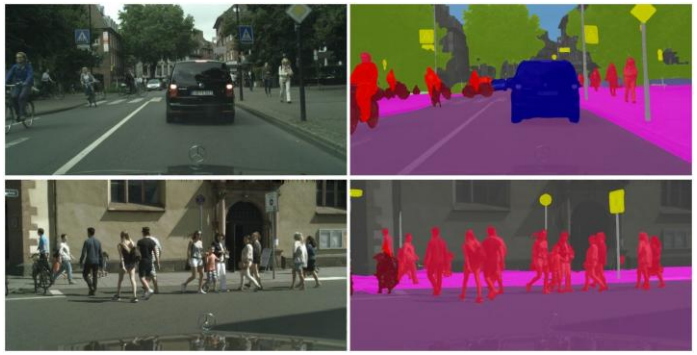


# What is Image Segmentation?

- **Definition:** The process of partitioning an image into multiple segments (regions/objects) where each pixel is assigned a label.
- **Goal:** Simplify the representation of an image into something more meaningful and easier to analyze.
- **Mathematical Definition:**
  - Partition image I into regions $R_1$, $R_2$, ..., $R_n$ such that:
    - $\cup R_i = I$ (covers whole image)
    - $R_i \cap R_j = \emptyset$ (non-overlapping)
    - $P(R_i)$ = TRUE (homogeneity criterion)





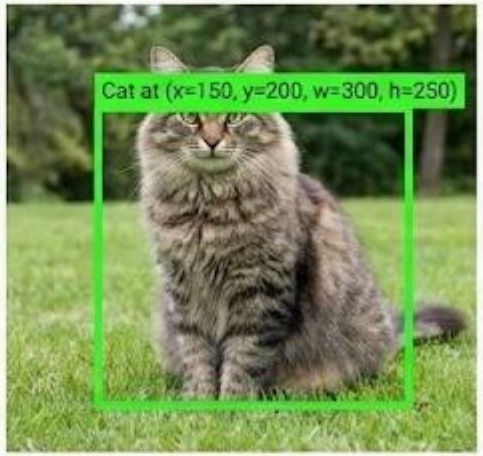
# Segmentation vs Detection vs Classification

**1. Original Image**

**2. Classification:**
"What's in the image?"


**Output:** Class label
"This is a cat"

**3. Detection:**
"Where are the objects?"




**Output:** Bounding boxes + labels

**4. Segmentation:**
"What is each pixel?"

**Output:** Pixel-wise masks
"These exact pixels are cat"

**Precision Levels**

- **Classification:** [Entire image] → "Cat"
- **Detection:** [Box around cat] → "Cat at coordinates"
- **Segmentation:** [Exact cat shape] → "These pixels = cat"

# Types of Segmentation

## 1. Semantic Segmentation

- Assigns class label to every pixel.
- Same class instances get same label.
- **Example:** All people = blue, all cars = red.

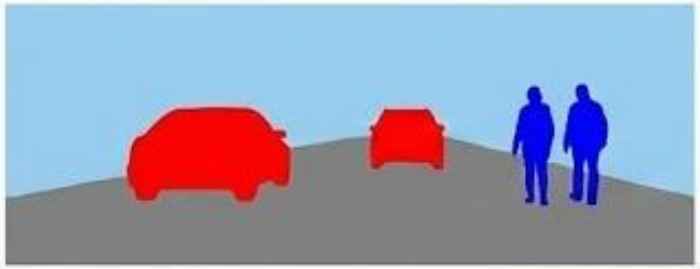
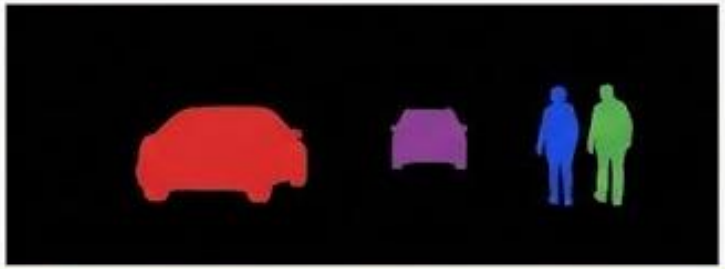
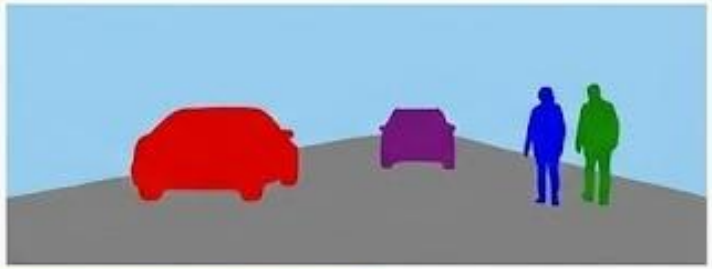
## 2. Instance Segmentation

- Assigns unique label to each object instance.
- Distinguishes between multiple objects of same class.
- **Example:** Person 1 = blue, Person 2 = green.

## 3. Panoptic Segmentation

- Combines semantic + instance.
- "Stuff" (sky, road) = semantic.
- "Things" (cars, people) = instance.

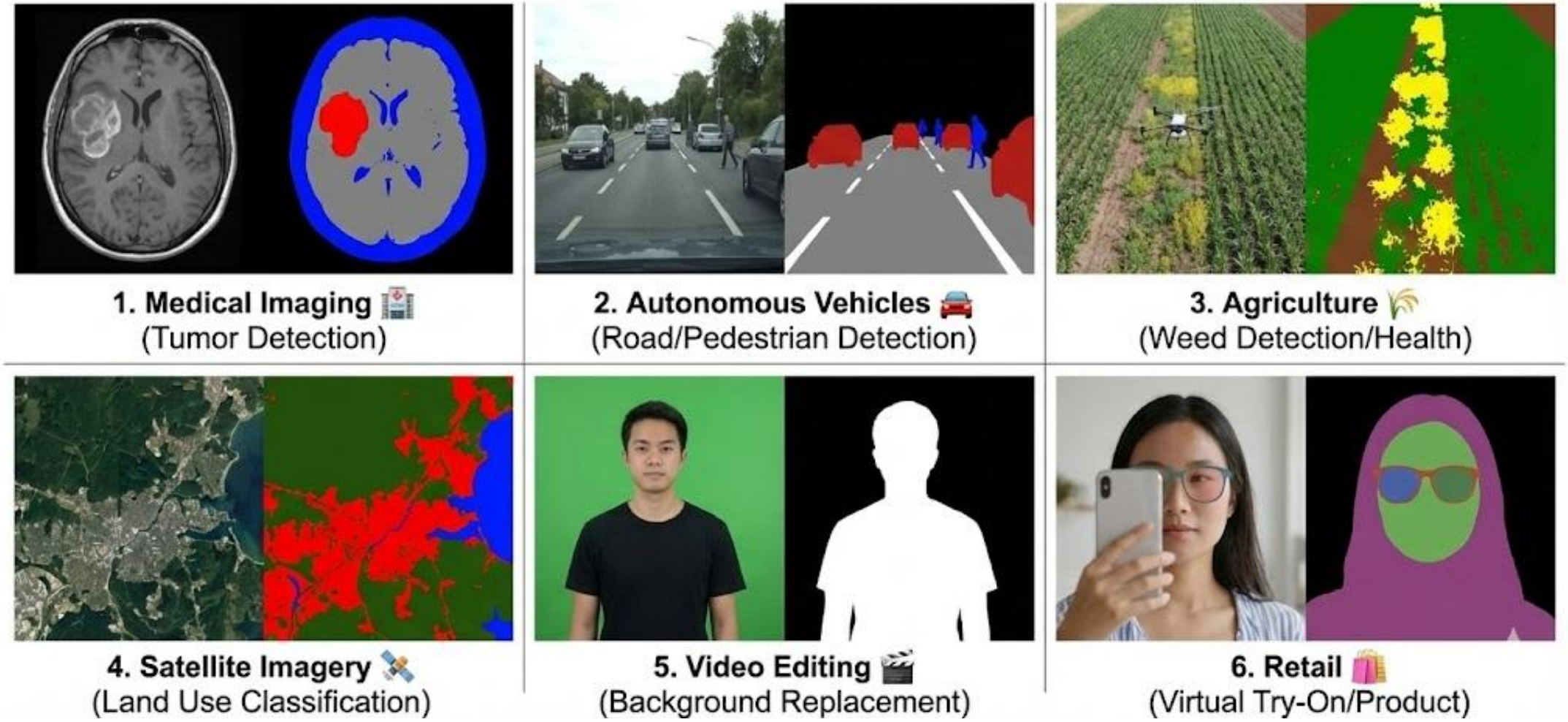


Comparison Table:

| Type | Question | Use Case |
|---|---|---|
| Semantic | "What class is each pixel?" | Scene understanding |
| Instance | "Which object instance is this?" | Object counting |
| Panoptic | "Complete scene understanding?" | Autonomous driving |

# Why Do We Need Segmentation?



# Challenges in Image Segmentation

**1. Ambiguous Boundaries**
(Fuzzy edges, gradual transitions)

**2. Occlusion**
(Objects hiding behind other objects,
Partial visibility)

**3. Illumination Variations**
(Shadows create false boundaries,
Highlights merge regions)

**4. Scale Variations**
(Same object at different sizes,
Near vs far objects)

**5. Intra-Class Variance**
(Dogs come in many shapes/colors,
All should be labeled "dog")


**6. Computational Cost**
(Pixel-level prediction is expensive,
Real-time requirements)

# Segmentation Quality - Evaluation Metrics

- **Why Evaluate?**
  - Segmentation isn't about pretty masks — it's about *overlap with truth*. These are the standard, no-nonsense metrics.

- **Pixel Accuracy**
  - **Definition:**
    Correct pixels ÷ Total pixels
  - **Issue:**
    Heavily biased toward large regions.
    (If background dominates, accuracy lies.)

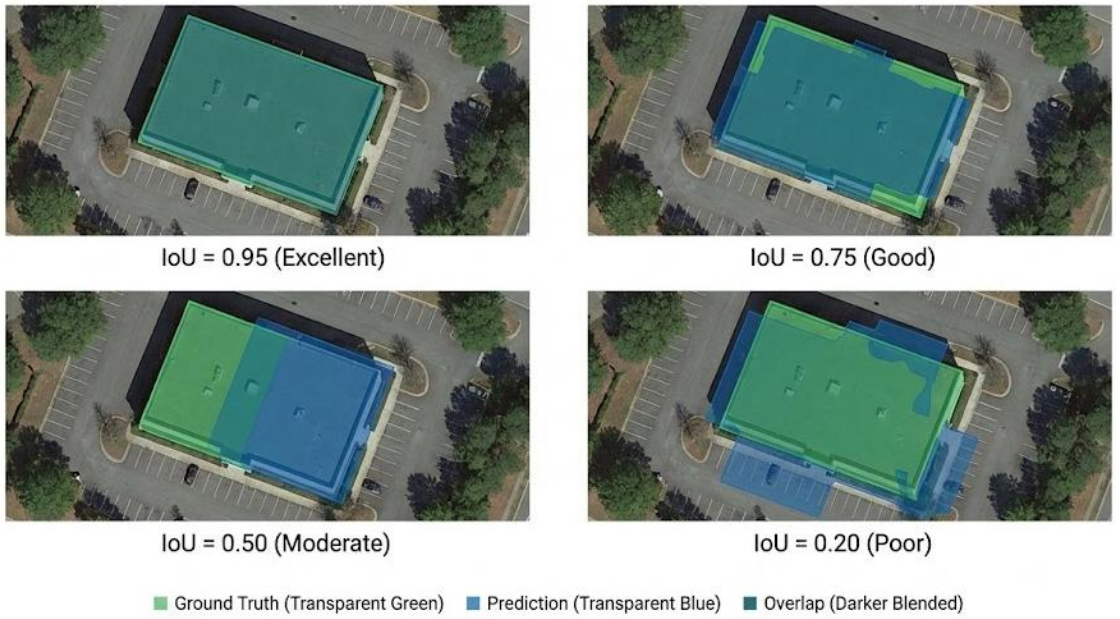
- **Intersection over Union (IoU / Jaccard)**

$$IoU = \frac{|Prediction \cap GT|}{|Prediction \cup GT|}$$

  - **Strength:**
    The most reliable metric for segmentation quality.

- **Dice Coefficient / F1 Score**

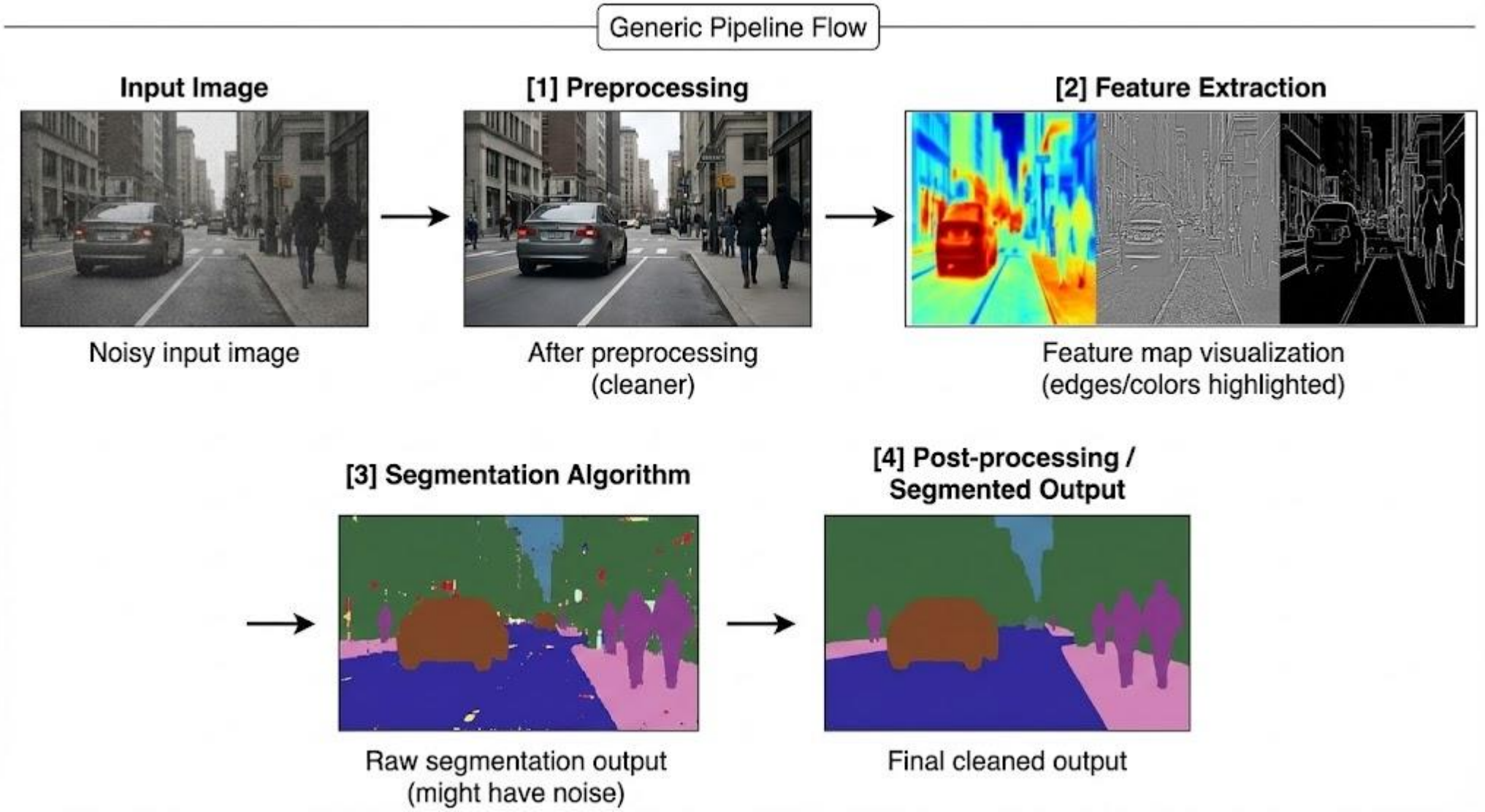
$$Dice = \frac{2|Prediction \cap GT|}{|Prediction| + |GT|}$$




**Mean IoU (mIoU)**

- Class-wise IoU, averaged.
- **Strength:**
  Best for multi-class segmentation benchmarks (Cityscapes, COCO, ADE20K).

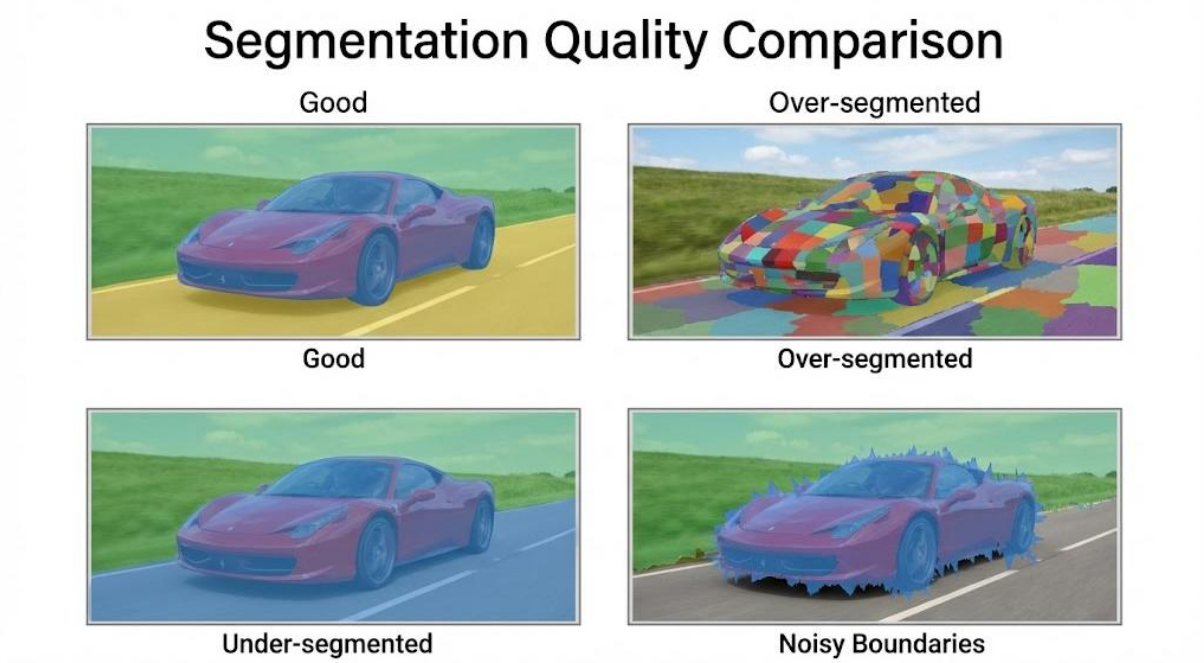
# The Segmentation Workflow



# Segmentation Criteria - What Makes a Good Segment?

## Homogeneity Criteria

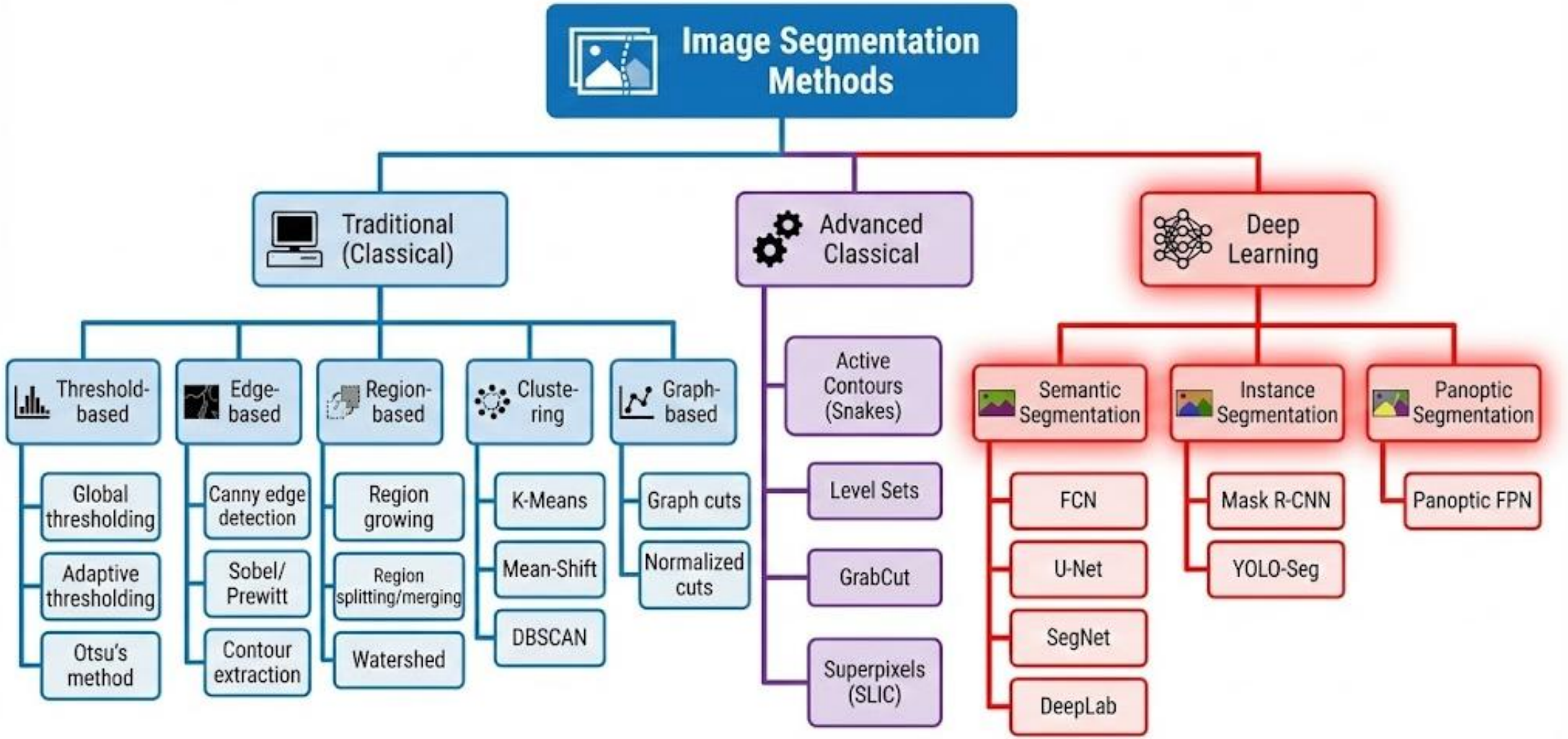
1. Intensity Similarity
2. Pixels in a region should have similar brightness.
3. Condition: variance σ(R) < threshold.
4. Color Similarity
5. Regions should contain pixels with similar color values (RGB / HSV).
6. Texture Similarity
7. Regions should share consistent texture patterns.
8. Spatial Connectivity
9. All pixels in a segment must be connected (4- or 8-connected).



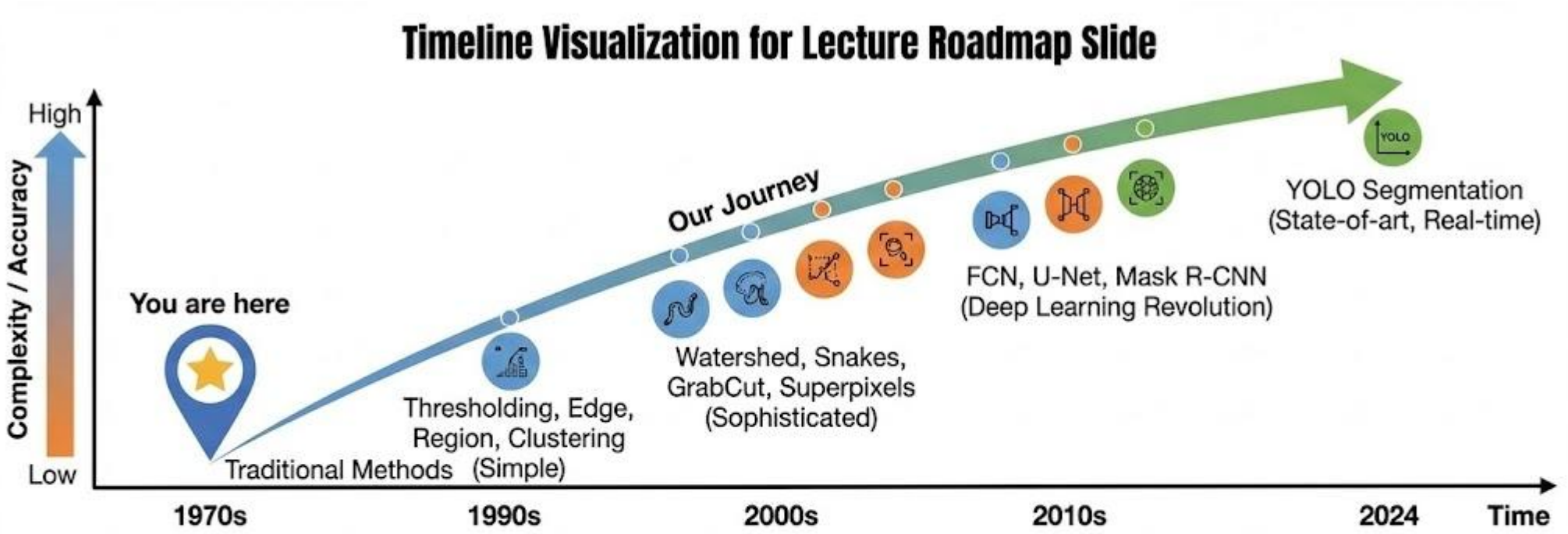

**Properties of a Good Segmentation**

- High similarity within regions
- High dissimilarity between regions
- Connected and meaningful regions
- Smooth, accurate boundaries

# Segmentation Taxonomy



# Lecture Roadmap




**Learning Progression** Understand principles → See limitations → Appreciate evolution → Master modern tools

| | Section | Methods | Key Takeaway |
|---|---|---|---|
| 1 | Part 2 | Thresholding, Edge, Region, Clustering | Foundation - simple but limited |
| 2 | Part 3 | Watershed, Snakes, GrabCut, Superpixels | More intelligent, still hand-crafted |
| 3 | Part 4 | FCN, U-Net, Mask R-CNN | Deep learning revolution |
| 4 | Part 5 | YOLO Segmentation | Real-time, practical deployment |

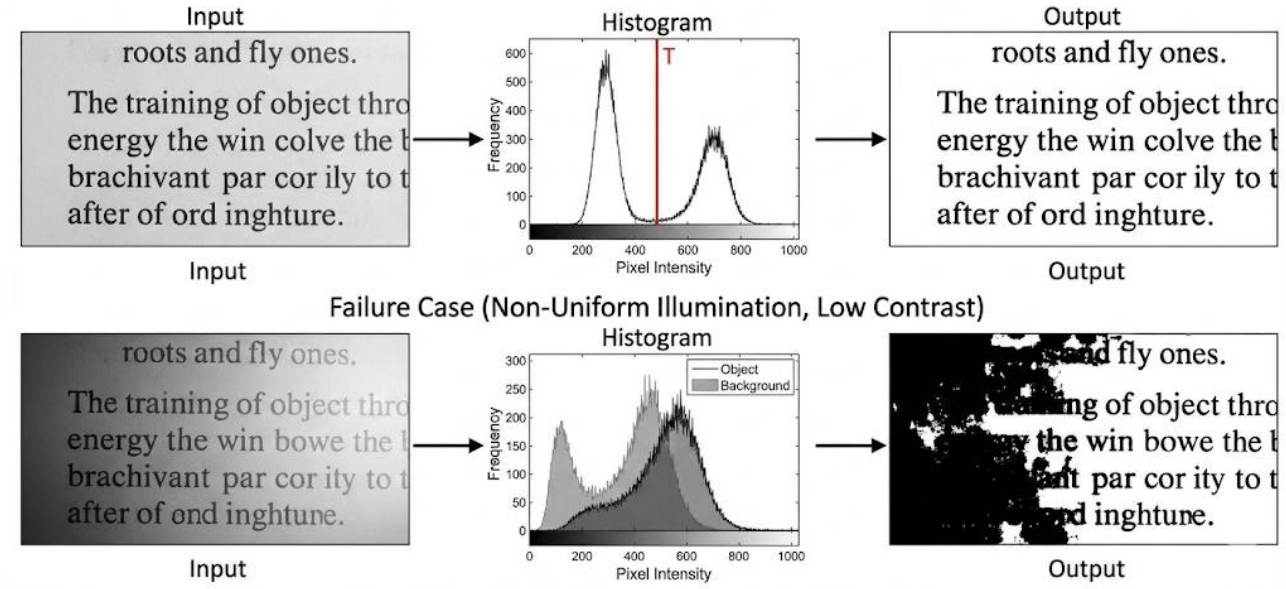
# Part 2 TRADITIONAL METHODS

## Thresholding - The Simplest Segmentation

- Separate objects from background based on intensity values.
- **Basic Formula:**
  - **Binary Segmentation**

$$g(x,y) = 1 \; if \; f(x,y) \geq T$$
$$0 \; if \; f(x,y) < T$$

  - Where:
  - f(x, y) = Input pixel intensity
  - T = Threshold value
  - g(x, y) = Output (1=object, 0=background)




When Does It Work?

- High contrast between object and background
- Uniform lighting
- Simple scenes (text, QR codes, outline)

When Does It Fail?

- Non-uniform illumination
- Low contrast
- Multiple objects with different intensities

# Global Thresholding Methods

## Manual Thresholding

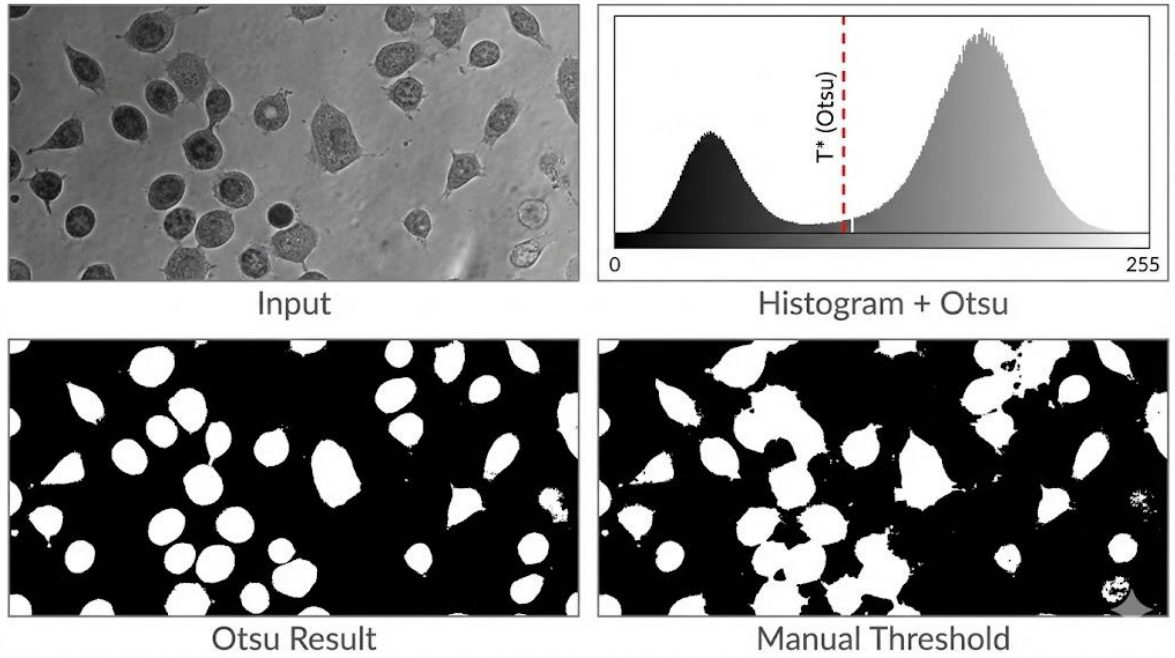
User selects a threshold value T manually.

Pros: Simple, easy to apply

Cons: Not automatic, subjective, fails under varying lighting

## Otsu's Method (1979)

Automatically finds the optimal threshold by maximizing the separation between two intensity classes.




## Algorithm

For each possible threshold T:

1. Split pixels into classes:
   - $C_0$: intensities < T
   - $C_1$: intensities ≥ T
2. Compute class means ($\mu_0$, $\mu_1$)
3. Compute class weights ($w_0$, $w_1$)
4. Compute between-class variance
5. Choose T that maximizes: $\sigma^2_{between} = w_0 \times w_1 \times (\mu_0 - \mu_1)^2$

**Assumptions of Otsu**

- Histogram is **bimodal** (two peaks)
- Classes have **reasonable balance**
- Image has **no strong illumination gradient**

# Adaptive Thresholding

**Problem with Global Thresholding:**

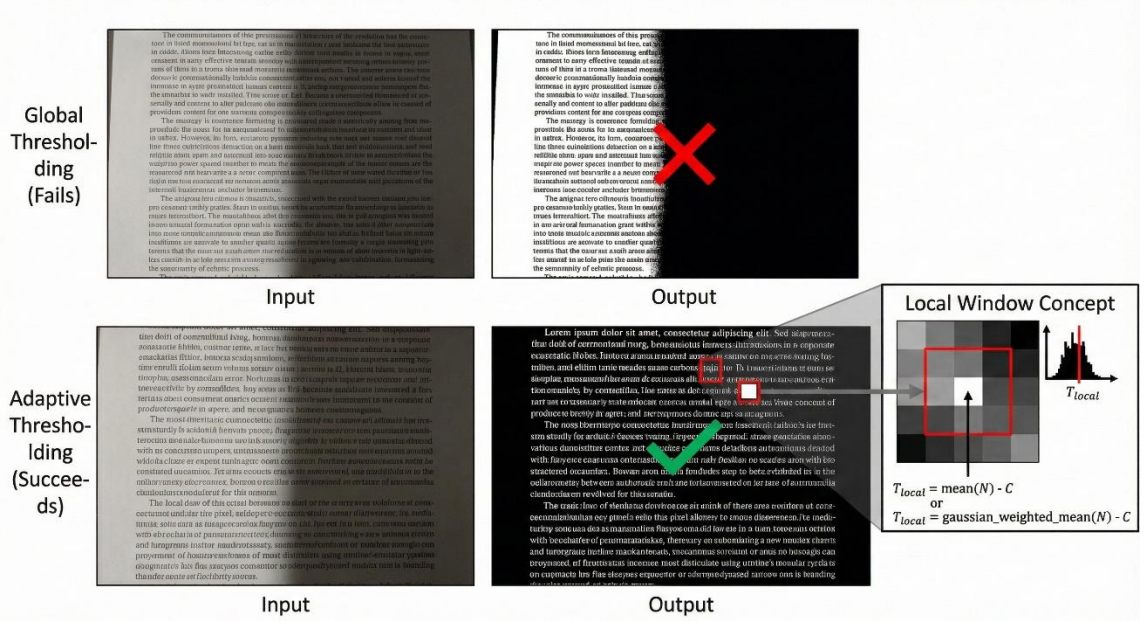
- Single threshold fails when lighting varies across the image

**Solution:**

- Calculate different thresholds for different regions of the image

**Algorithm:**

For each pixel (x, y):
1. Define neighborhood N (e.g., 15×15 window)
2. Calculate local threshold $T_{local}$ from N
3. Apply threshold: g(x, y) = 1 if f(x, y) > $T_{local}$




**Two Variants:**

- **Mean Adaptive Thresholding**

$$T_{local} = mean(N) - C$$

- **Gaussian Adaptive Thresholding**

$$T_{local} = gaussian\_weighted\_mean(N) - C$$

**Parameters:**
**Block size:** Neighborhood window size (11, 15, 21...)
**C value:** Constant subtracted from mean (2-10)

# Multi-Level Thresholding

## Beyond Binary:

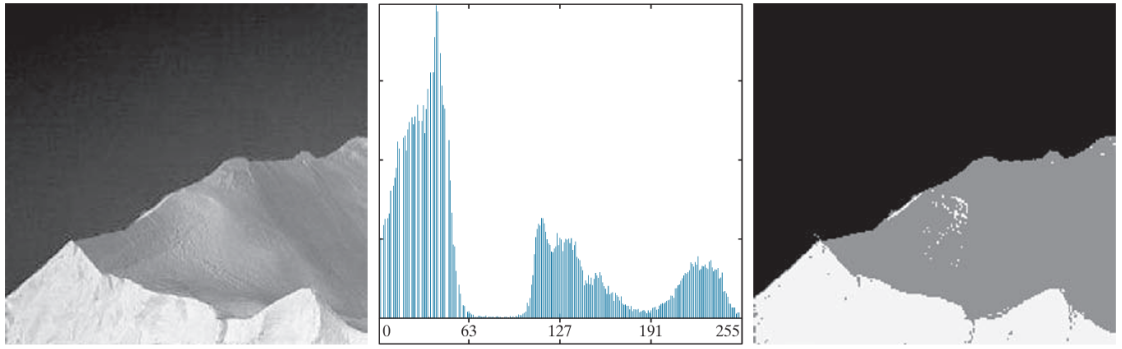
- What if we have multiple objects with different intensities?

## Approach:

- Use **multiple thresholds** to create multiple segments

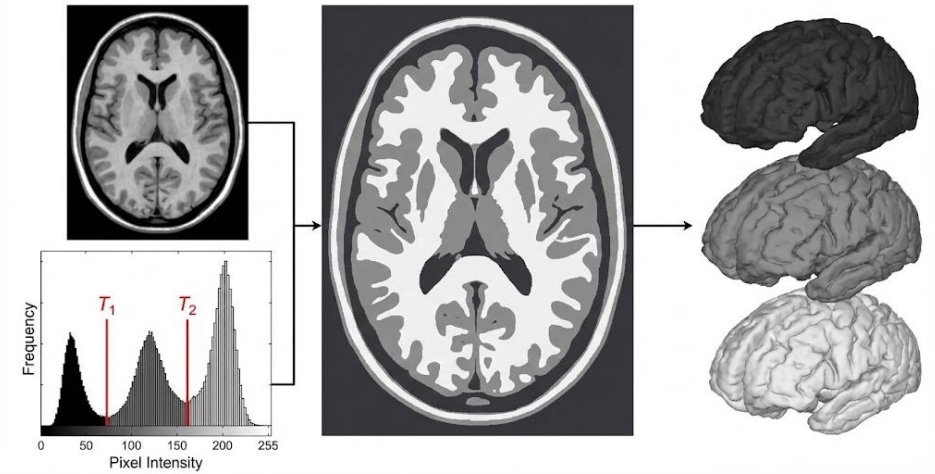
## Formula:

$$
g(x, y) = \begin{cases} 0 & \text{if } f(x, y) < T_1 \\ 1 & \text{if } T_1 \le f(x, y) < T_2 \\ 2 & \text{if } T_2 \le f(x, y) < T_3 \\ \dots & \\ n & \text{if } f(x, y) \ge T_n \end{cases}
$$




a b c

**FIGURE 10.42** (a) Image of an iceberg. (b) Histogram. (c) Image segmented into three regions using dual Otsu thresholds. (Original image courtesy of NOAA.)




## Finding Multiple Thresholds:

- **Otsu's method extended:** Find thresholds that maximize total inter-class variance
- **Valley detection:** Find histogram valleys between peaks
- **Clustering:** K-Means on intensity histogram

# Edge-Based Segmentation

**Different Approach:**

- Instead of looking at pixel intensities, look at **intensity changes (edges)**

**Core Assumption:**

- Object boundaries = Strong intensity gradients

**Process:**

1. Detect edges
2. Link edge fragments into contours
3. Fill contours to create regions

Gradient Operators:
∂f/∂x (horizontal changes)
∂f/∂y (vertical changes)

Gradient magnitude: $\sqrt{((\partial f/\partial x)^2 + (\partial f/\partial y)^2)}$
Gradient direction: $\tan^{-1}(\partial f/\partial y \,/\, \partial f/\partial x)$

| | | |
|---|---|---|
| $z_1$ | $z_2$ | $z_3$ |
| $z_4$ | $z_5$ | $z_6$ |
| $z_7$ | $z_8$ | $z_9$ |

| | | | |
|---|---|---|---|
| −1 | 0 | 0 | −1 |
| 0 | 1 | 1 | 0 |

Roberts

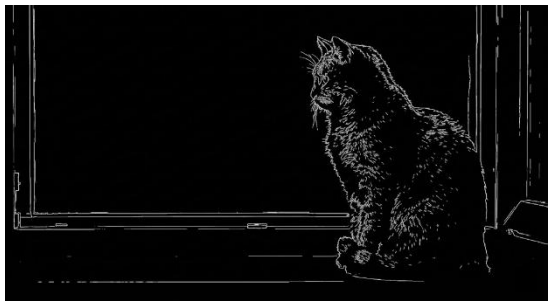
| | | | | | |
|---|---|---|---|---|---|
| −1 | −1 | −1 | −1 | 0 | 1 |
| 0 | 0 | 0 | −1 | 0 | 1 |
| 1 | 1 | 1 | −1 | 0 | 1 |

Prewitt

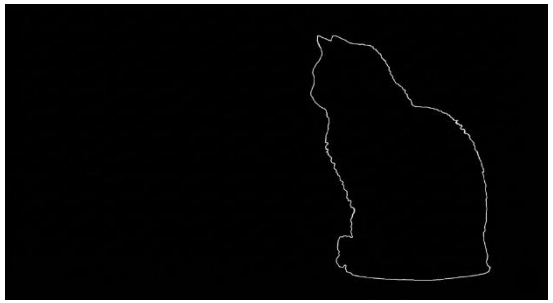
| | | | | | |
|---|---|---|---|---|---|
| −1 | −2 | −1 | −1 | 0 | 1 |
| 0 | 0 | 0 | −2 | 0 | 2 |
| 1 | 2 | 1 | −1 | 0 | 1 |

Sobel

Original Image



Detected Edges



Link edges into contour

Fill contours to create edges

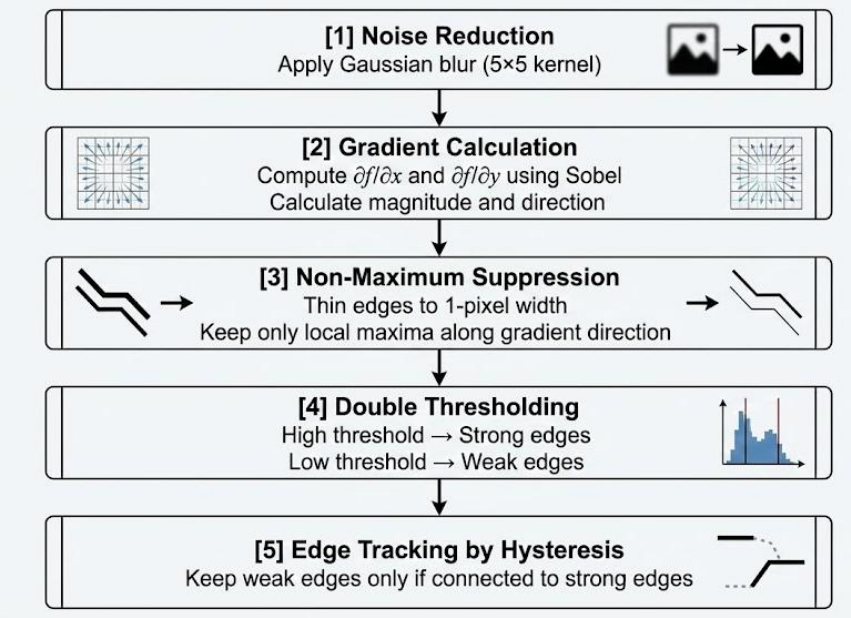
# Canny Edge Detection

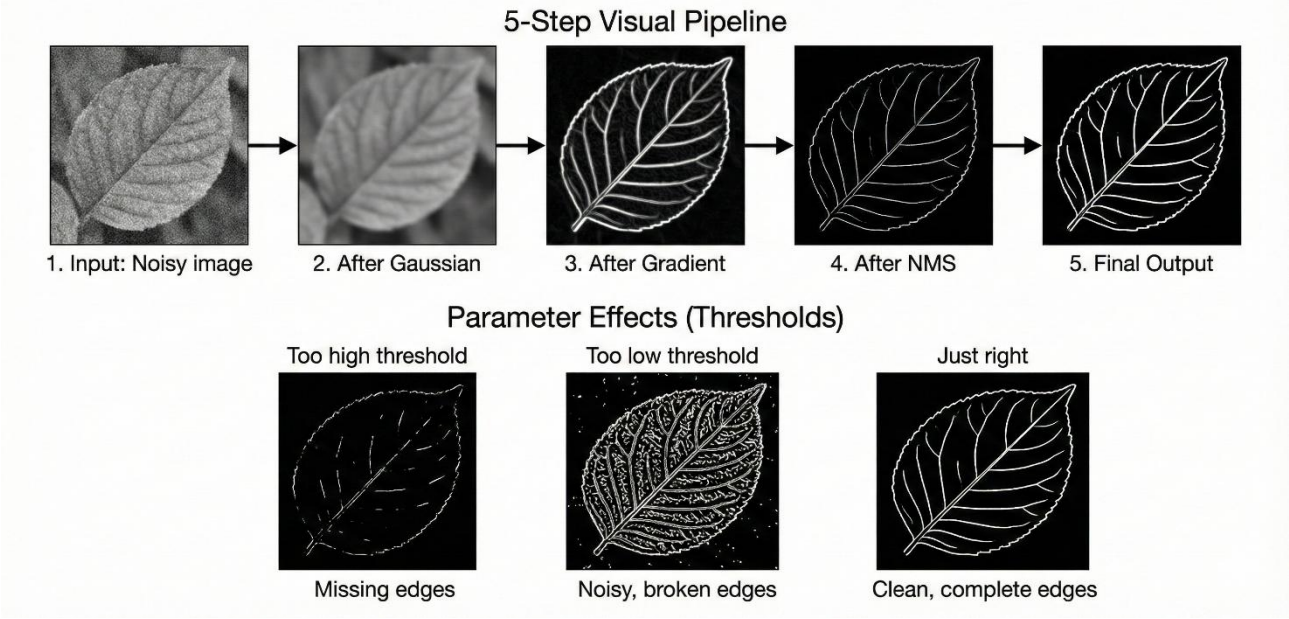
## Why Canny is Special:

## Optimizes three criteria:

1. **Good detection:** Low error rate
2. **Good localization:** Edges close to true position
3. **Single response:** One detection per edge

## Algorithm Steps:



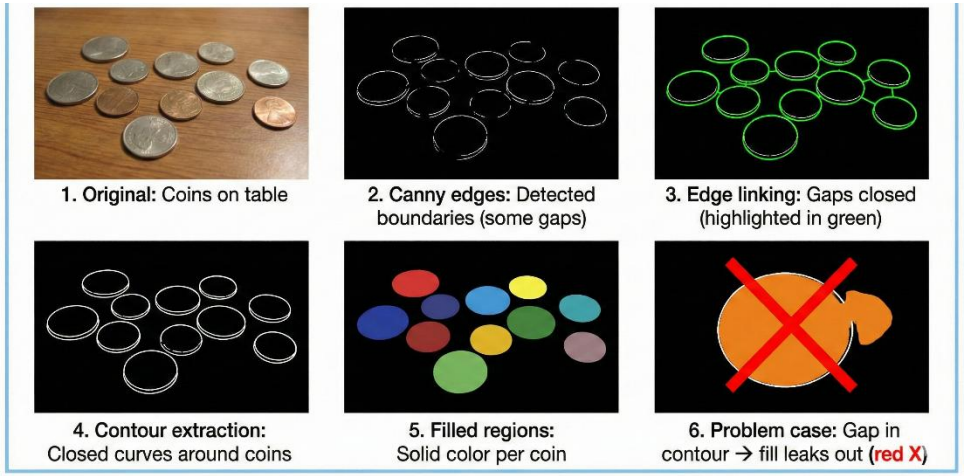
# From Edges to Regions

**The Gap Problem:**

- Edge detection gives us lines, but we need **filled regions**

**Contour-Based Segmentation:**

- Edge Detection → Contour Extraction → Region Filling




Step 1: Edge Linking
Connect broken edge fragments:

- **Proximity criterion:** Distance < threshold
- **Direction criterion:** Similar gradient direction
- **Hough transform:** Find parametric curves (lines, circles)

Step 2: Contour Extraction
Find closed boundaries:

- **Chain codes:** Trace boundary as sequence of moves
- **Active contours:** Evolve curve to fit boundary (coming later!)

Step 3: Region Filling
Fill interior of contours:

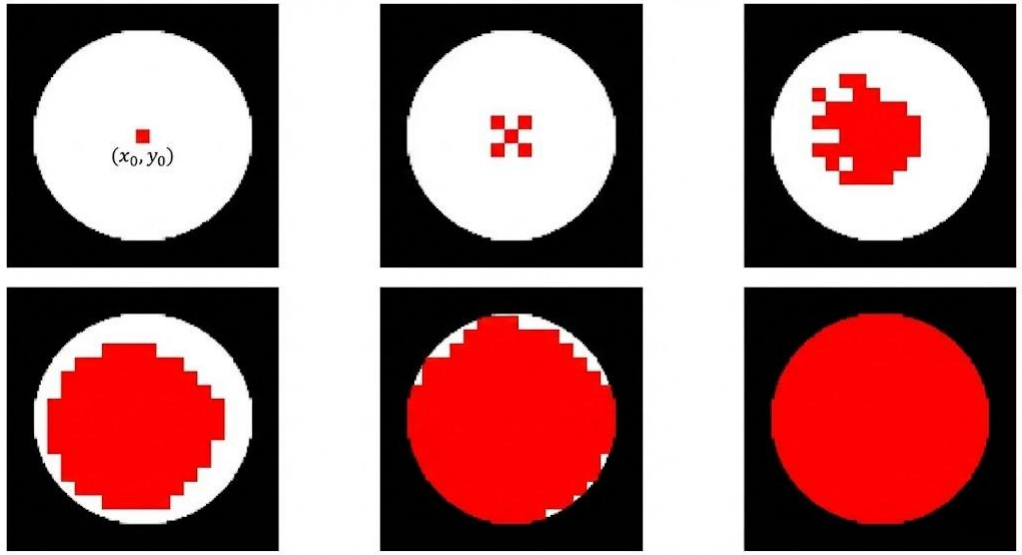
- **Flood fill algorithm**
- **Scan-line fill**

Problems:

- **Open contours:** Gaps prevent filling
- **Over-segmentation:** Too many small regions
- **Nested contours:** Holes inside objects

# Region-Based Segmentation - Region Growing

**Philosophy:**
Start from seed points and **grow regions** by adding similar neighbors




**Algorithm:**
Input: Image I, seed point ($x_0$, $y_0$), similarity criterion T

1. Initialize:
   Region R = {($x_0$, $y_0$)}
   Queue Q = neighbors of ($x_0$, $y_0$)

2. While Q not empty:
   a. Pop pixel (x, y) from Q
   b. If similar to R (based on criterion T):
      - Add (x, y) to R
      - Add neighbors of (x, y) to Q

3. Output: Region R

**Similarity Criteria:**
1. Intensity Difference:

$$|I(x,y) - mean(R)| < T$$

Color Distance (RGB/Lab):

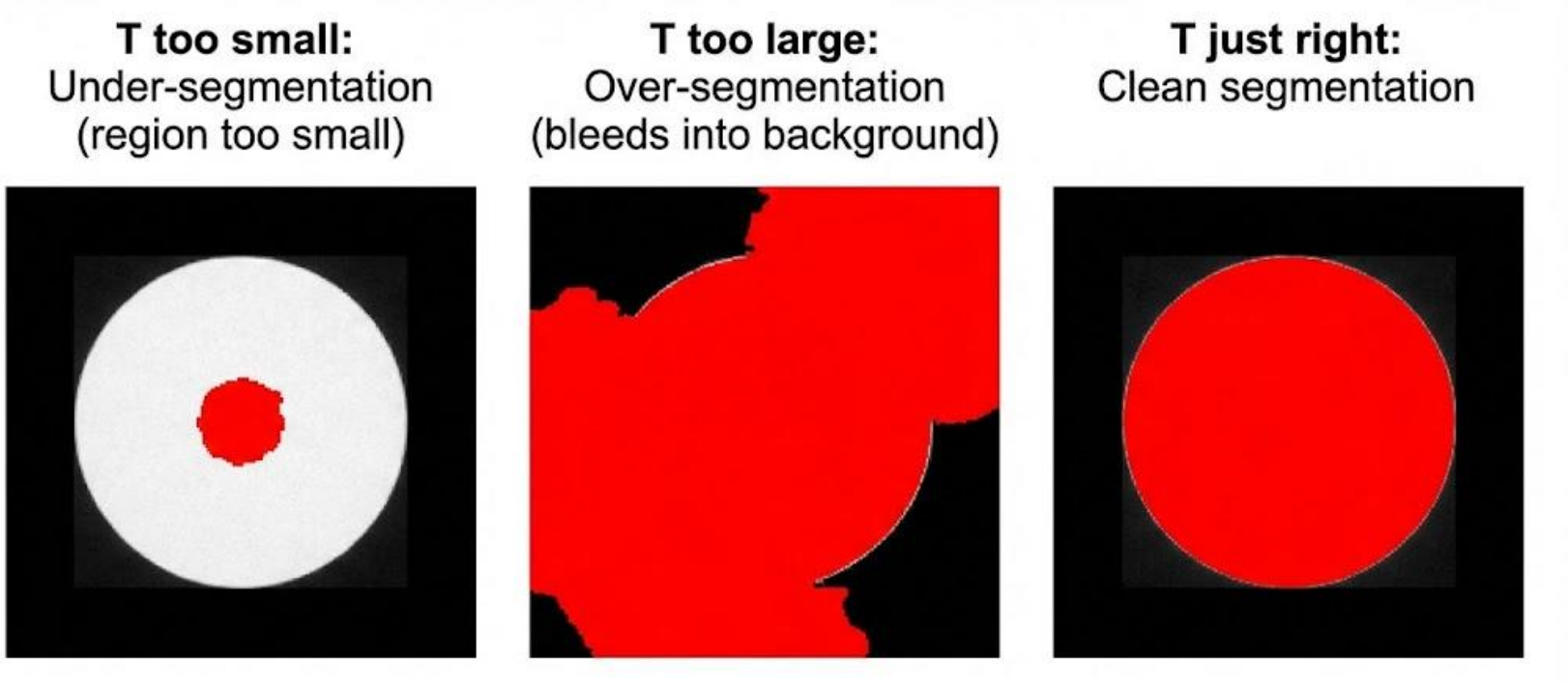
$$||color(x,y) - mean\_color(R)|| < T$$

Texture Similarity:

$$texture\_distance(x,y,R) < T$$

# Region-Based Segmentation - Region Growing

**Philosophy:**
Start from seed points and **grow regions**
by adding similar neighbors




**Seed Selection:**
- Manual: User clicks
- Automatic: Grid of seeds, then merge
- Interest points: Harris corners, SIFT key points

# Region Splitting and Merging

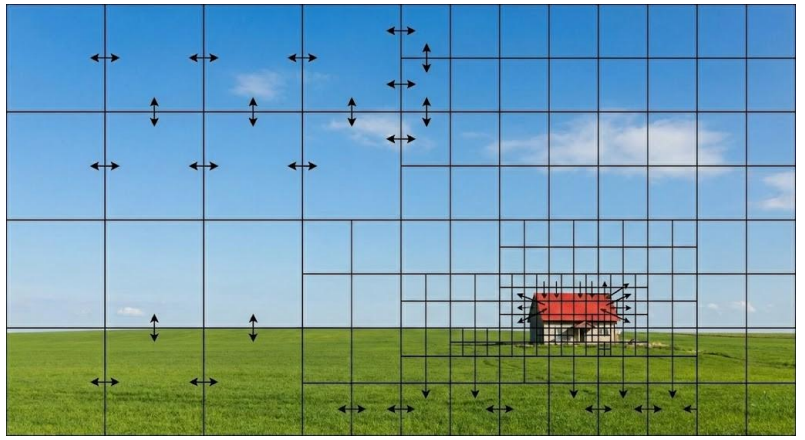
**Dual Approach:**
Combine **top-down splitting** with **bottom-up merging**

**Algorithm:**
Phase 1: SPLITTING
1. Start with entire image as one region
2. For each region R:
   If R is not homogeneous:
      Split R into 4 quadrants (quadtree)
      Recursively check sub-regions

Phase 2: MERGING
3. For each pair of adjacent regions ($R_1$, $R_2$):
   If $R_1$ and $R_2$ are similar:
      Merge $R_1$ and $R_2$

4. Repeat until no more merging possible

**Homogeneity Test:**
Region R is homogeneous if:
- Standard deviation $\sigma(R) <$ threshold
- OR: $|\max(R) - \min(R)| <$ threshold
- OR: Texture uniformity > threshold



# Clustering-Based Segmentation - K-Means

**New Perspective:**

- Treat each pixel as a **data point in feature space**, then cluster

**Algorithm:**
Input: N pixels, K clusters

1. Initialize K cluster centers randomly

2. Repeat until convergence:

   a. Assignment step:
      For each pixel:
         Assign to nearest cluster center

   b. Update step:
      For each cluster:
         Recompute center as mean of assigned pixels

3. Output: Cluster labels for each pixel


Distance Metric:
$d(\text{pixel}, \text{center}) = \sqrt{(\Sigma(f_i - c_i)^2)}$

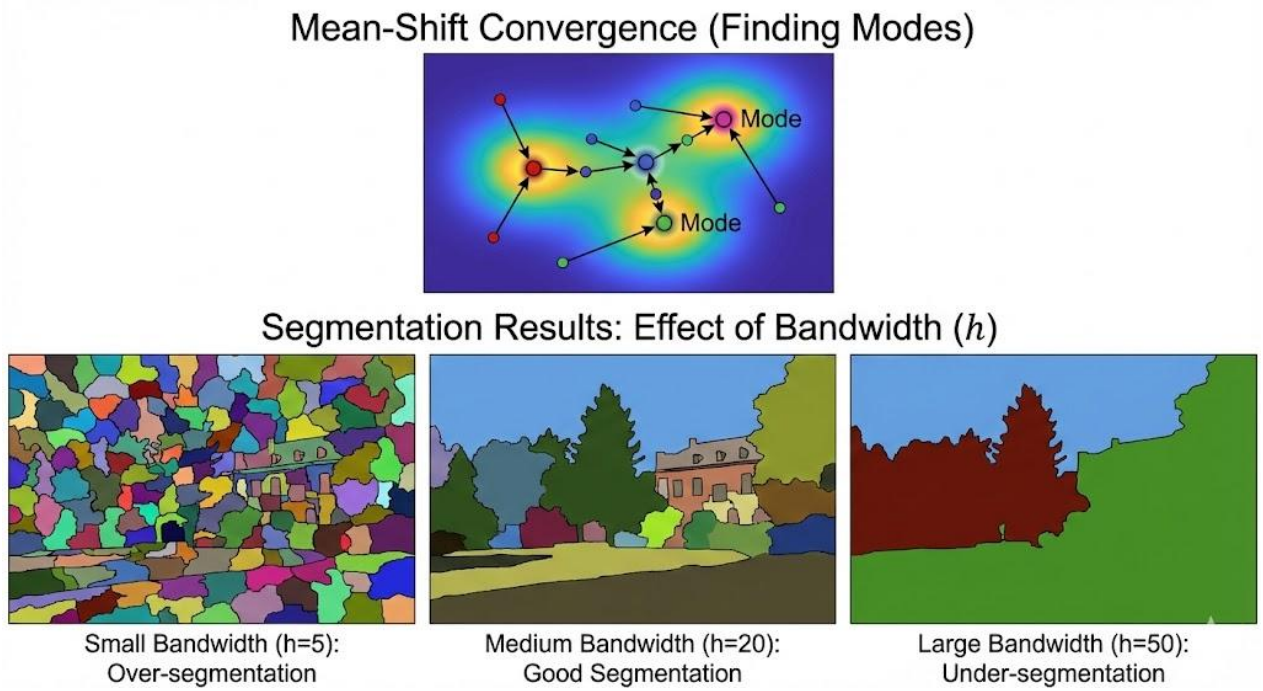
# Mean-Shift Clustering

**Advantage Over K-Means:**

- **Automatically finds number of clusters!**
  No need to specify K

Core Idea:

- Each data point "climbs" to nearest mode (peak) in density distribution




**Algorithm:**

For each pixel:

1. Define window W around pixel
2. Compute mean of all points in W: m
3. Shift window center to m
4. Repeat until convergence
5. Pixel belongs to cluster at final window position

The Mean-Shift Vector:

Direction: Current position → Mean of neighborhood

Magnitude: Proportional to density gradient

# Graph-Based Segmentation - Introduction

**New Representation:**

- Image as **weighted graph G = (V, E)**

**Vertices (V):**

- Each pixel = vertex

**Edges (E):**

- Connect adjacent pixels (4 or 8-connectivity)

**Edge Weights (w):**

- Similarity/dissimilarity between pixels

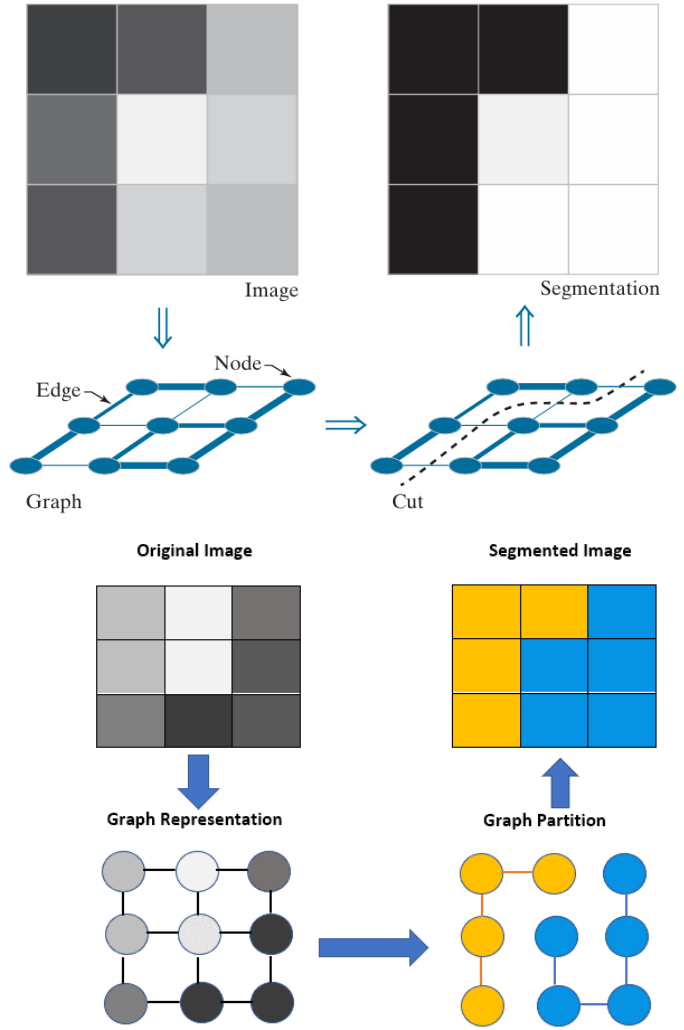
$w(i,j) = |I(i) - I(j)|$ $(intensity\ difference)$

OR

$w(i,j) = ||color(i) - color(j)||$ $(color\ distance)$

**Segmentation as Graph Partitioning:**

- Find cuts in graph that:
  - **Minimize** edges cut (similarity within regions)
  - **Maximize** dissimilarity between regions



# Graph-Based Segmentation - Introduction

**Graph Cut Objective:**

$$Cut(A, B) = \Sigma\, w(i, j) \;\; for\; i \in A, j \in B$$

Goal: Minimize Cut while balancing region sizes

**Two Main Approaches:**

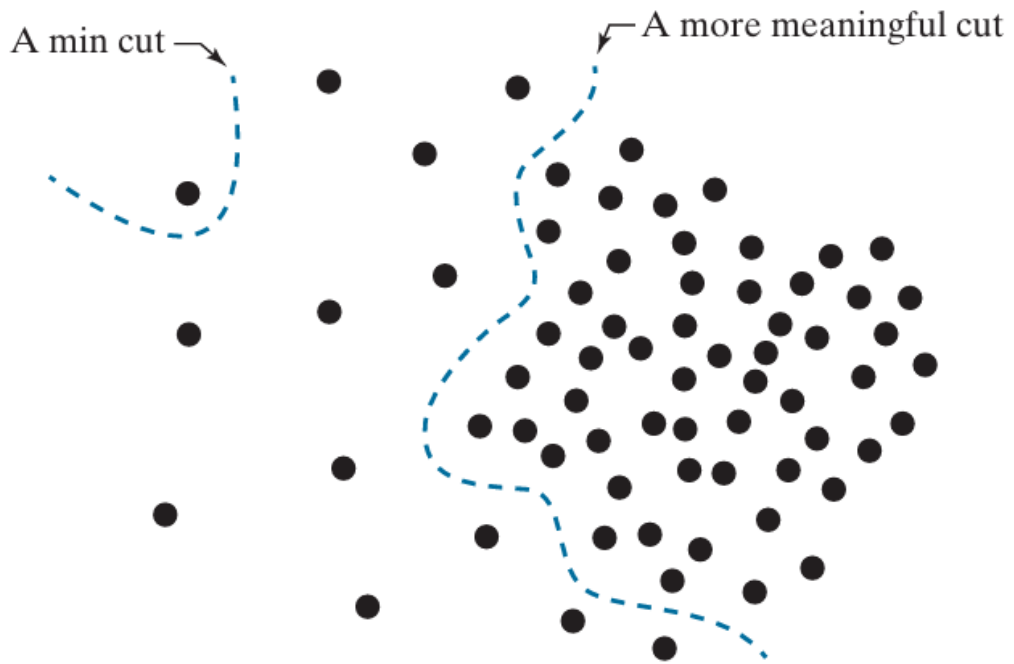
**1. Minimum Cut**

- Problem: Prefers tiny regions

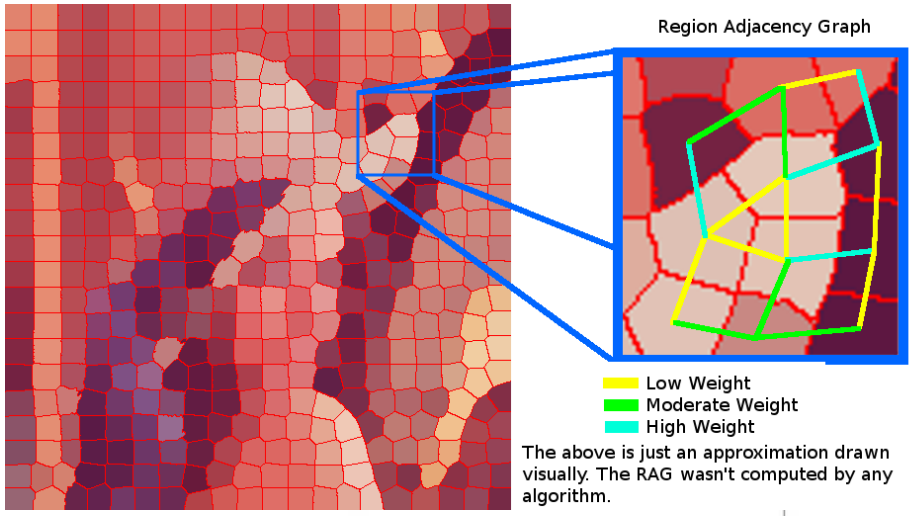
**2. Normalized Cut**

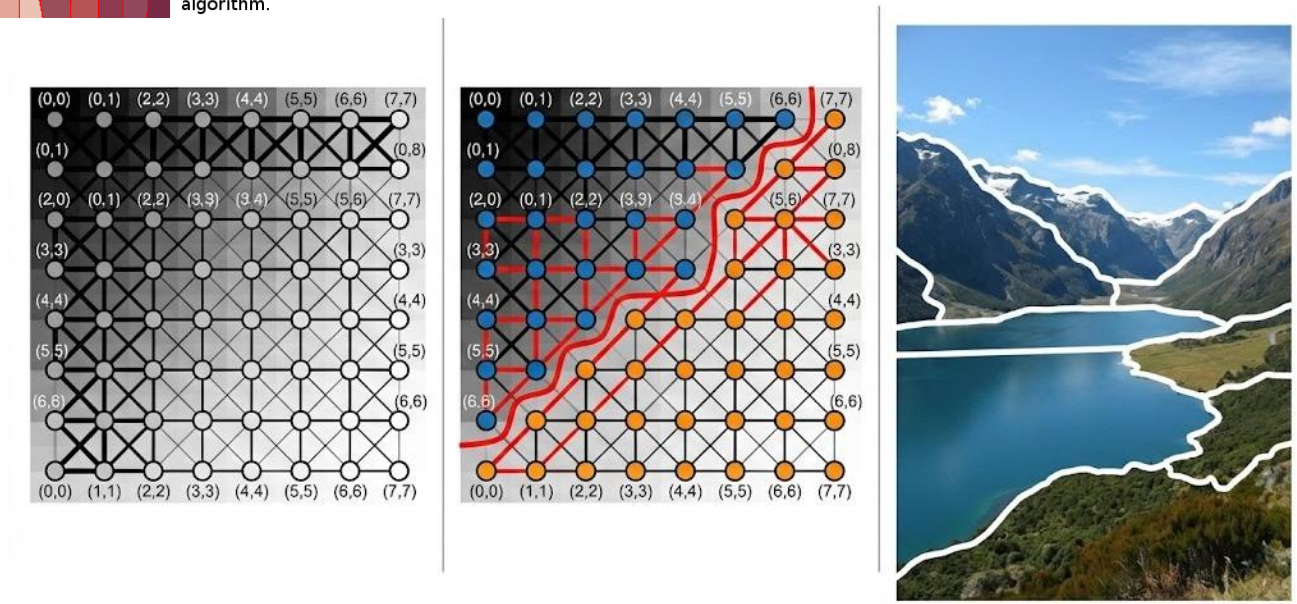
- Balances cut value with region sizes

- $NCut(A, B) = \frac{Cut(A,B)}{Vol(A)} + \frac{Cut(A,B)}{Vol(B)}$



# Graph-Based Segmentation - Example



# Traditional Methods - Limitations

**Hand-Crafted Features**

- Based on human intuition (intensity, color, edges)
- May not capture semantic information
- Different scenes need different features

Low-Level Vision Only

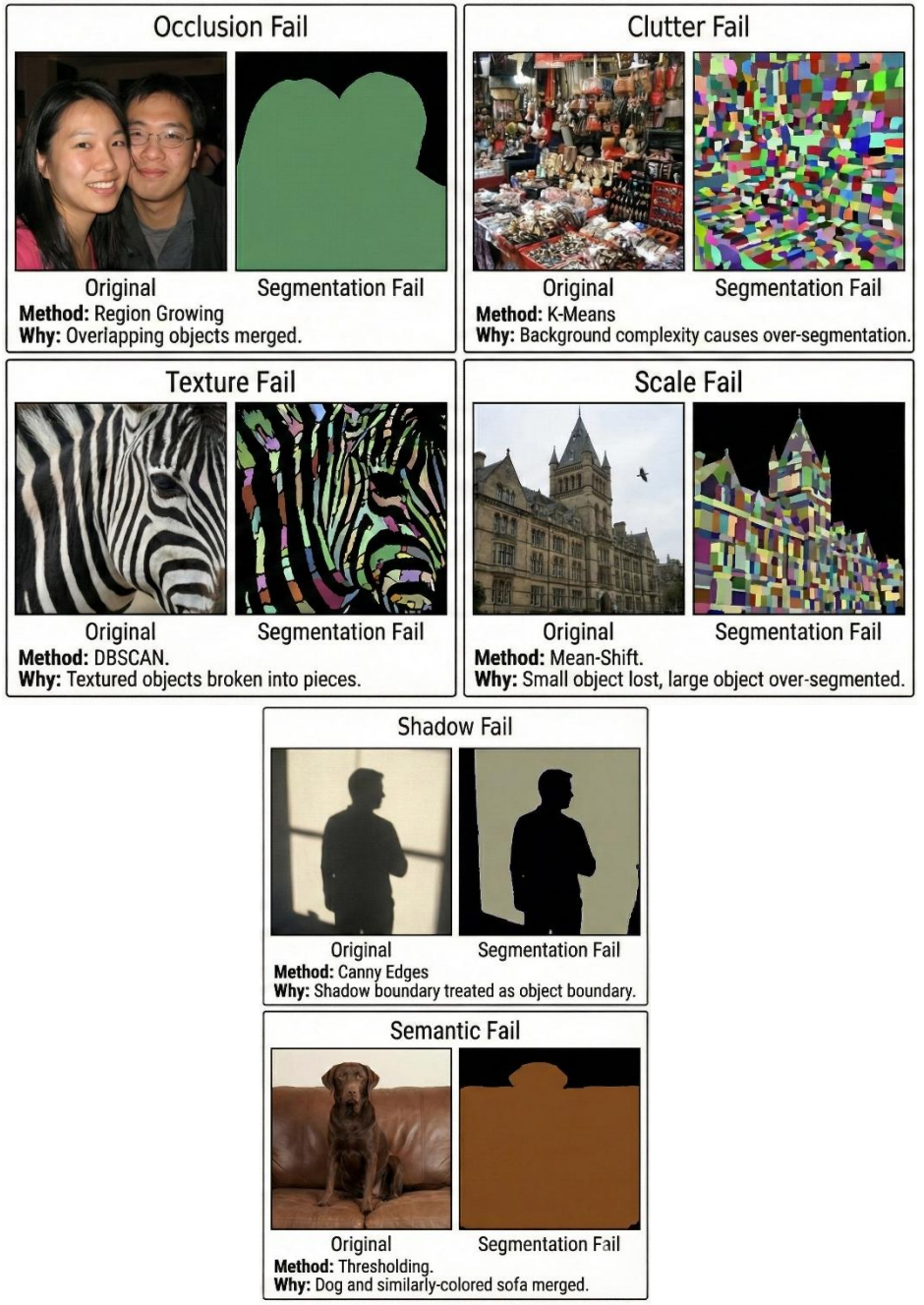
- Operate on pixels, not objects
- No concept of "what" they're segmenting
- Can't distinguish "dog" from "cat" if similar color

Semantic Gap

- Methods see colors/textures
- Humans see objects/categories
- Example: Two people of different skin tones → segmented separately

No Learning

- Can't improve with more data
- Can't adapt to new domains



# PART 3: ADVANCED CLASSICAL METHODS

## Watershed Algorithm

Imagine the image as a **topographic surface** where:

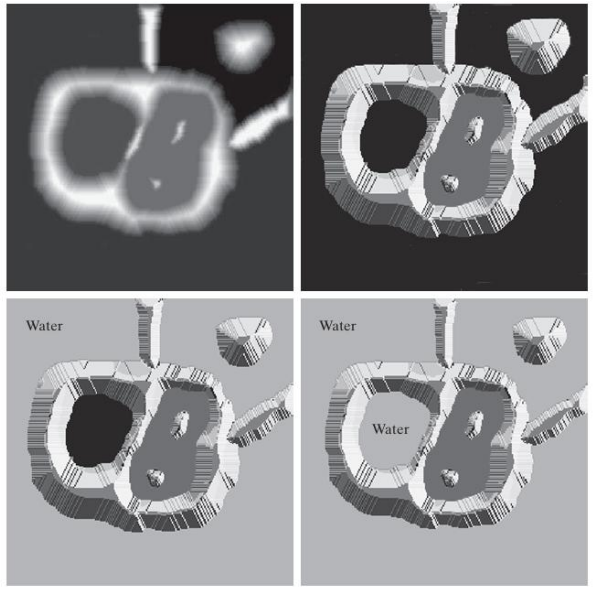
- Height = Gradient magnitude
- Valleys = Regions
- Ridges = Boundaries

The Flooding Process:

- 1. Poke holes at regional minima (lowest points)
- 2. Gradually flood the surface
- 3. When waters from different basins meet → Build dam
- 4. Dams = Segmentation boundaries

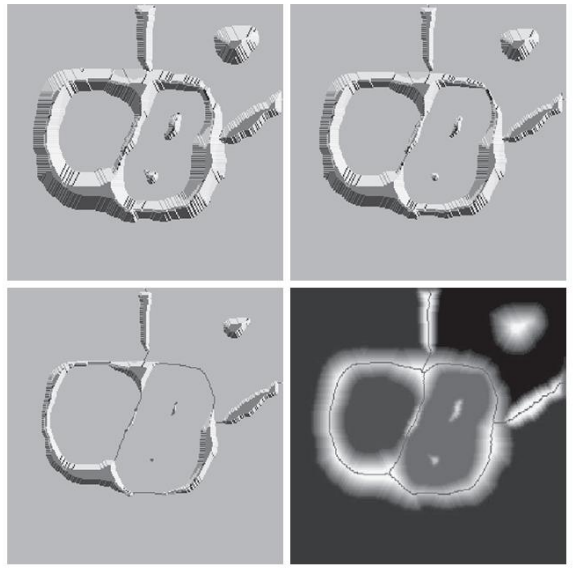
Why "Watershed"?

- Boundaries are like watershed lines in geography (dividing rivers into basins)




a c
b d

**FIGURE 10.57**
(a) Original image.
(b) Topographic view. Only the background is *black*. The basin on the left is slightly lighter than black.
(c) and (d) Two stages of flooding. All constant dark values of gray are intensities in the original image. Only constant *light gray* represents "water."
(Courtesy of Dr. S. Beucher, CMM/ Ecole des Mines de Paris.)
(*Continued on next page.*)



e f
g h

**FIGURE 10.57**
*(Continued)*
(e) Result of further flooding.
(f) Beginning of merging of water from two catchment basins (a short dam was built between them).
(g) Longer dams.
(h) Final watershed (segmentation) lines superimposed on the original image.
(Courtesy of Dr. S. Beucher, CMM/ Ecole des Mines de Paris.)

# Watershed Algorithm

Immersion Process:

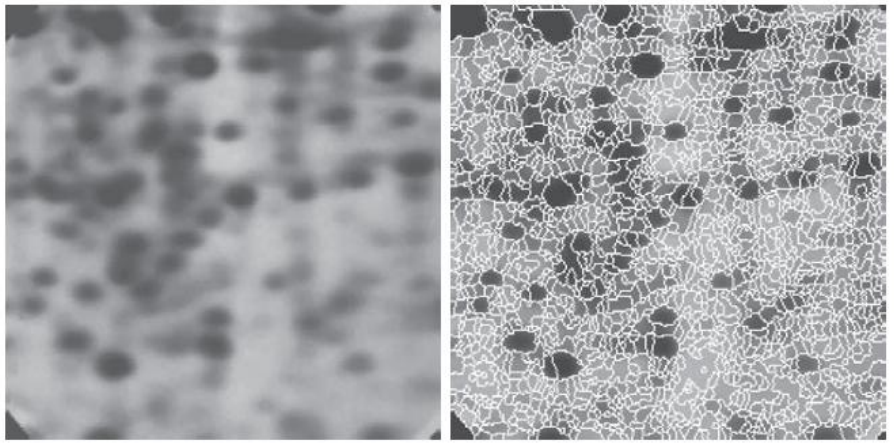
- At each level h:
- Expand existing catchment basins
- Create new basins at new minima
- Build watershed at basin intersections

Applications:

- Cell counting in microscopy
- Separating touching objects
- Segmenting convex objects

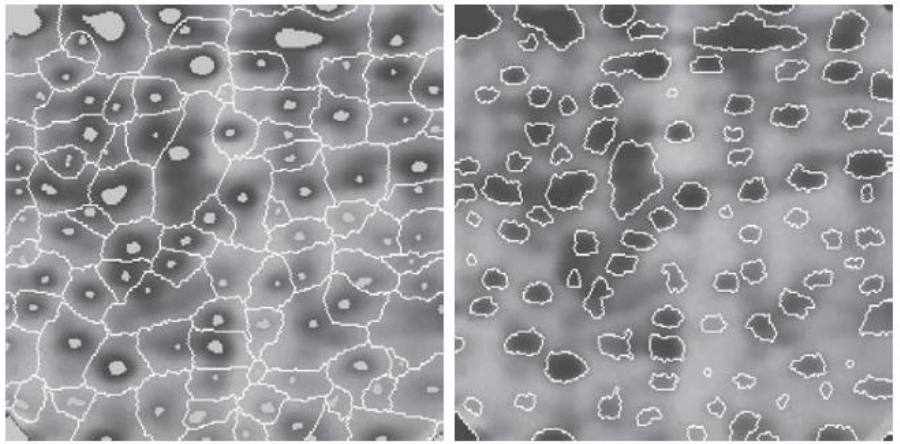
a b

**FIGURE 10.60**
(a) Electrophoresis image. (b) Result of applying the watershed segmentation algorithm to the gradient image. Over-segmentation is evident. (Courtesy of Dr. S. Beucher, CMM/ Ecole des Mines de Paris.)



a b

**FIGURE 10.61**
(a) Image showing internal markers (light gray regions) and external markers (watershed lines). (b) Result of segmentation. Note the improvement over Fig. 10.60(b). (Courtesy of Dr. S. Beucher, CMM/ Ecole des Mines de Paris.)



# Active Contours (Snakes)

## What It Is

- A curve that moves across the image and adjusts itself to lock onto an object's boundary.

## How It Works

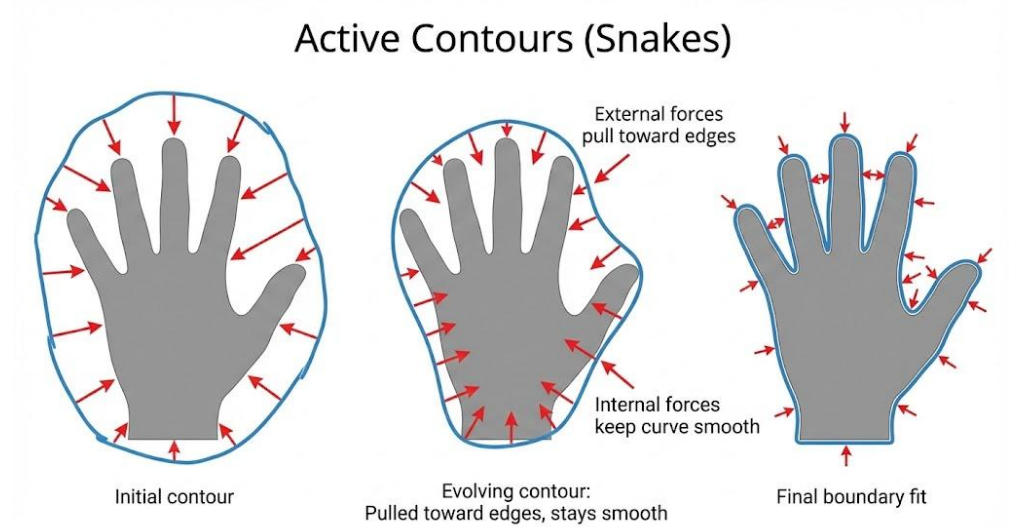
- The curve tries to stay smooth and stable.
- At the same time, it is pulled toward strong image features like edges.
- It keeps adjusting its position until it settles on the object outline.

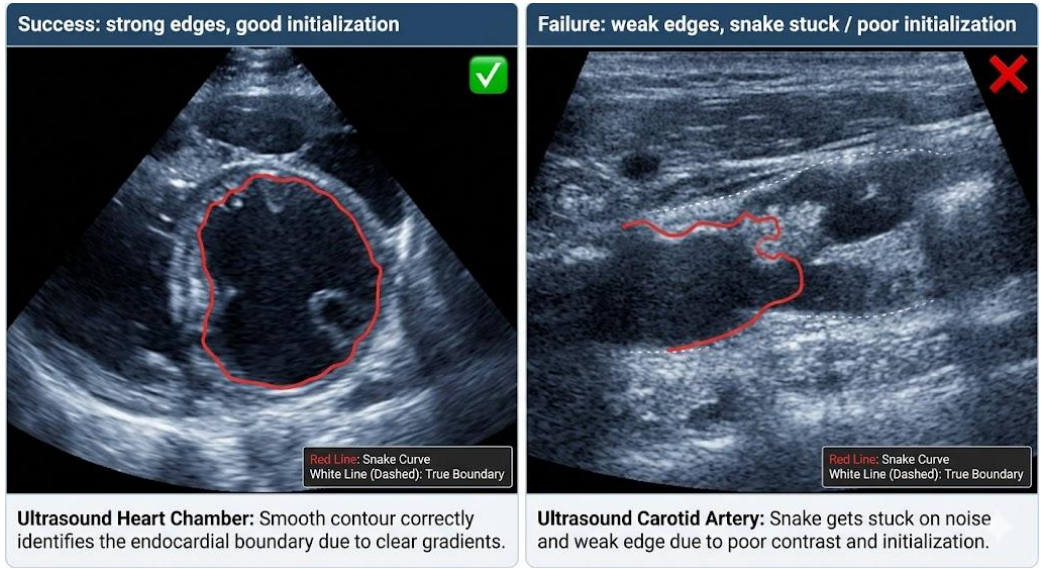
## Why It's Useful

- Produces smooth, natural-looking boundaries
- Can incorporate prior shape expectations
- Offers very precise boundary localization

## Limitations

- Must start close to the target object
- Can get stuck before reaching the real boundary
- Needs careful tuning of parameters



# Active Contours — Variants

## Classic Snakes (1987)

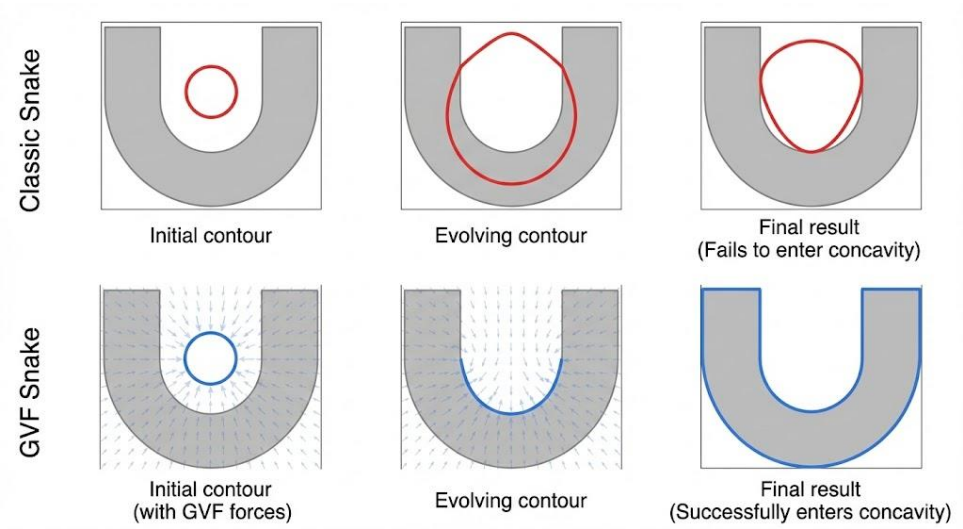
- Use a manually initialized curve that moves toward object edges
- Works well only when the starting curve is already close to the boundary
- Poor performance in deep concavities or complex shapes

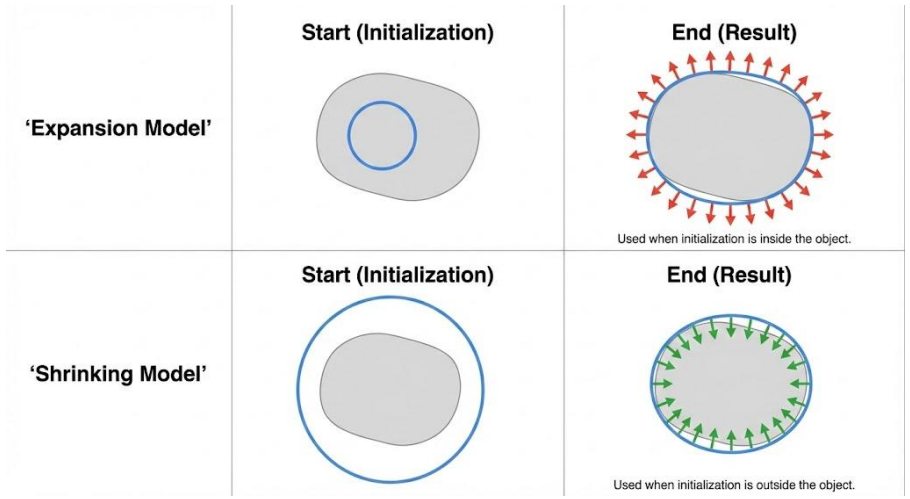
## GVF Snakes (Gradient Vector Flow)

- Designed to fix classic snake's short capture range
- Spreads edge information across the entire image so the curve can be attracted from far away
- Able to move into narrow concave regions that classic snakes miss

## Balloon Models

- Add a force that makes the curve expand or shrink like a balloon
- Expansion helps when starting inside an object
- Shrinking helps when starting outside



# Geometric Active Contours - Level Set Methods

## Idea

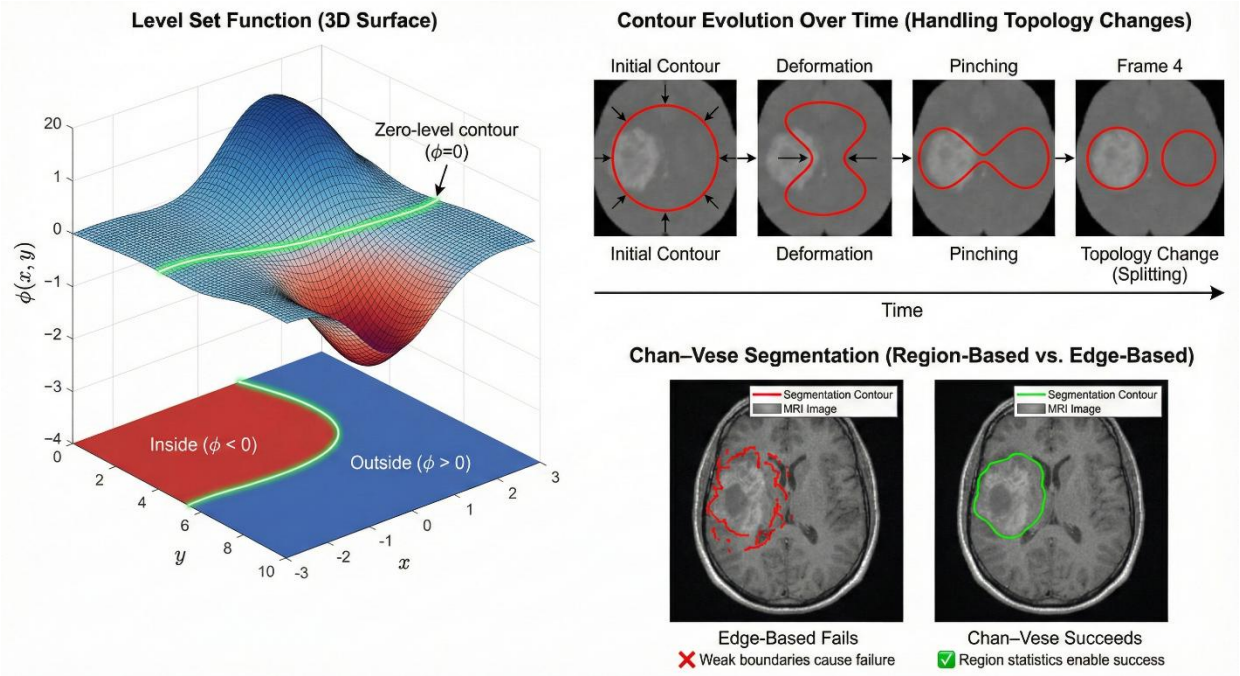
- Represent the contour indirectly using a function instead of tracking curve points.
- The object boundary is simply where this function crosses zero.

## How It Works (Conceptually)

- Inside the object → function is negative
- Outside the object → function is positive
- The boundary forms naturally where the function transitions between them

## Why This Representation Is Powerful

- The contour can split or merge automatically (no manual handling)
- No need to keep track of curve points
- Extends cleanly to 3D and higher dimensions




## Region-Based Variant (Chan–Vese Model)

- Doesn't rely on strong edges
- Uses region homogeneity instead of gradients
- Works well on low-contrast or fuzzy images (e.g., MRI)

# GrabCut — Interactive Segmentation

## Goal

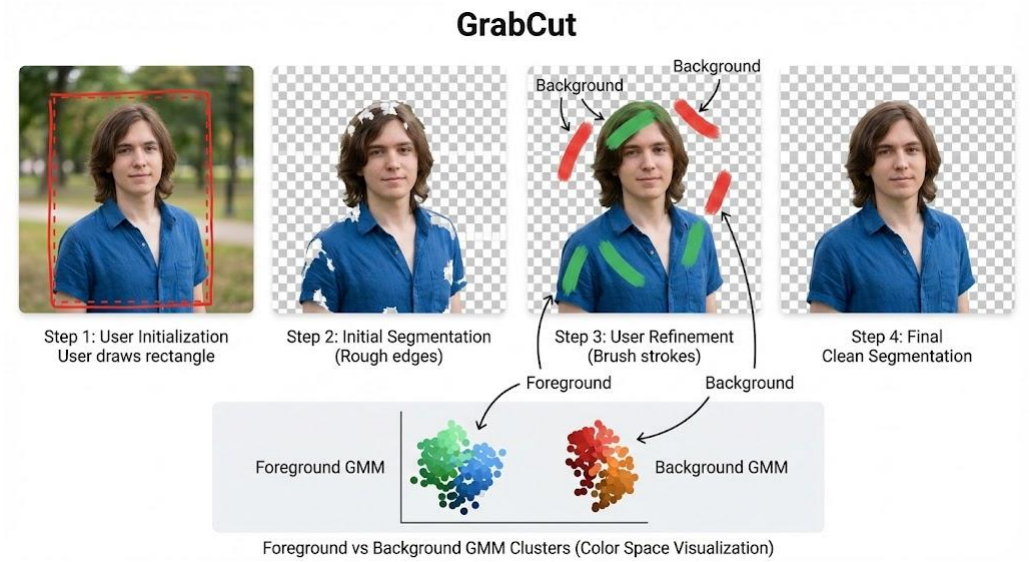
- Segment objects using very little user input.

## How It Works (Conceptually)

1. User gives a rough hint (a rectangle or a few scribbles).
2. The algorithm marks clear background, clear foreground, and uncertain regions.
3. It builds color models for foreground and background.
4. A graph-cut step separates the object from the background.
5. The system updates the color models and repeats until the result stabilizes.
6. User can add a few strokes if the result needs improvement.

## Why It's Effective

- Requires only a loose box or minimal scribbles
- Automatically refines the segmentation
- Works well even with imperfect initialization
- Produces high-quality cutouts




## User Interaction

- Draw rectangle → gives the algorithm a starting point
- Optional strokes → correct problem areas
- Algorithm refines → segmentation becomes clean and accurate

# Superpixels — Perceptual Grouping

## Why Superpixels?

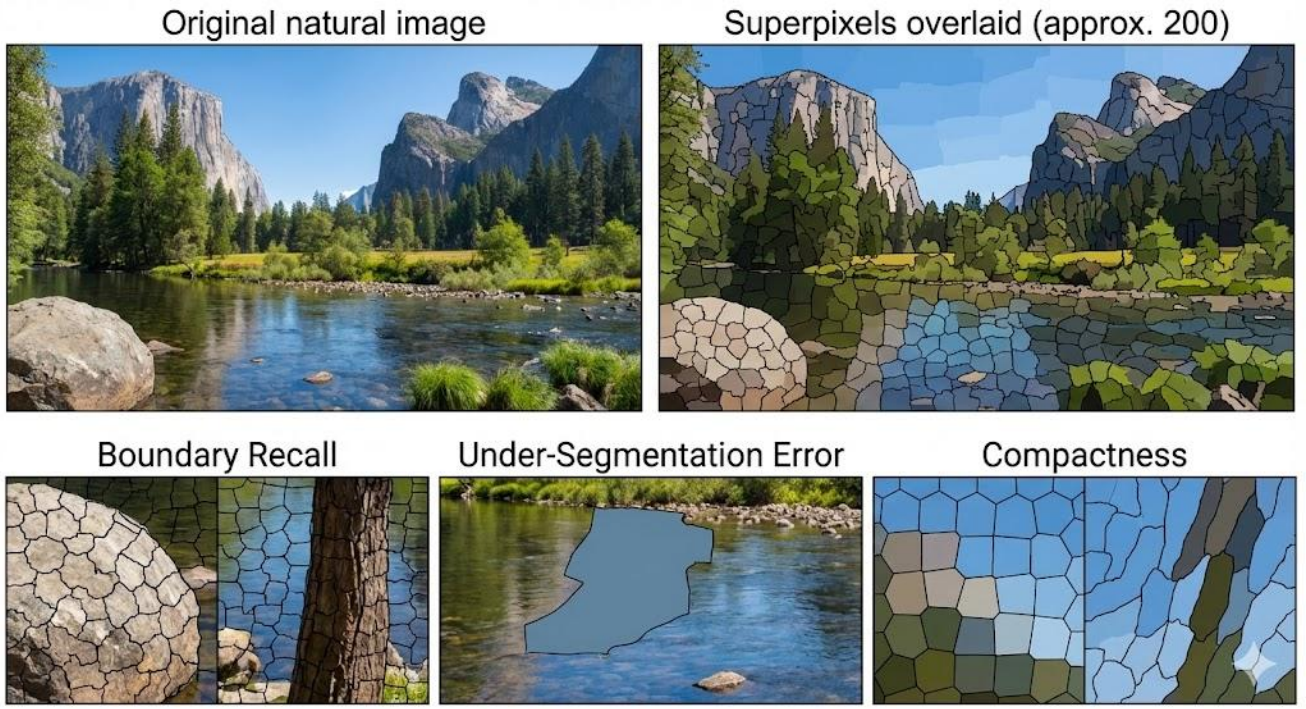
- Pixels are too small, noisy, and meaningless individually. Grouping them into larger, coherent regions makes processing simpler and more aligned with how we perceive images.

## What They Are

- Clusters of nearby pixels that share similar color/texture, forming small, meaningful regions.

## Why They're Useful

- Reduce tens of thousands of pixels into a few hundred regions
- Respect natural boundaries in the image
- Much faster to process than raw pixels
- Provide richer, more stable features for segmentation or detection




**What Good Superpixels Should Have**

- Follow real object boundaries
- Be compact and not stretched or messy
- Have roughly consistent size
- Be quick to compute

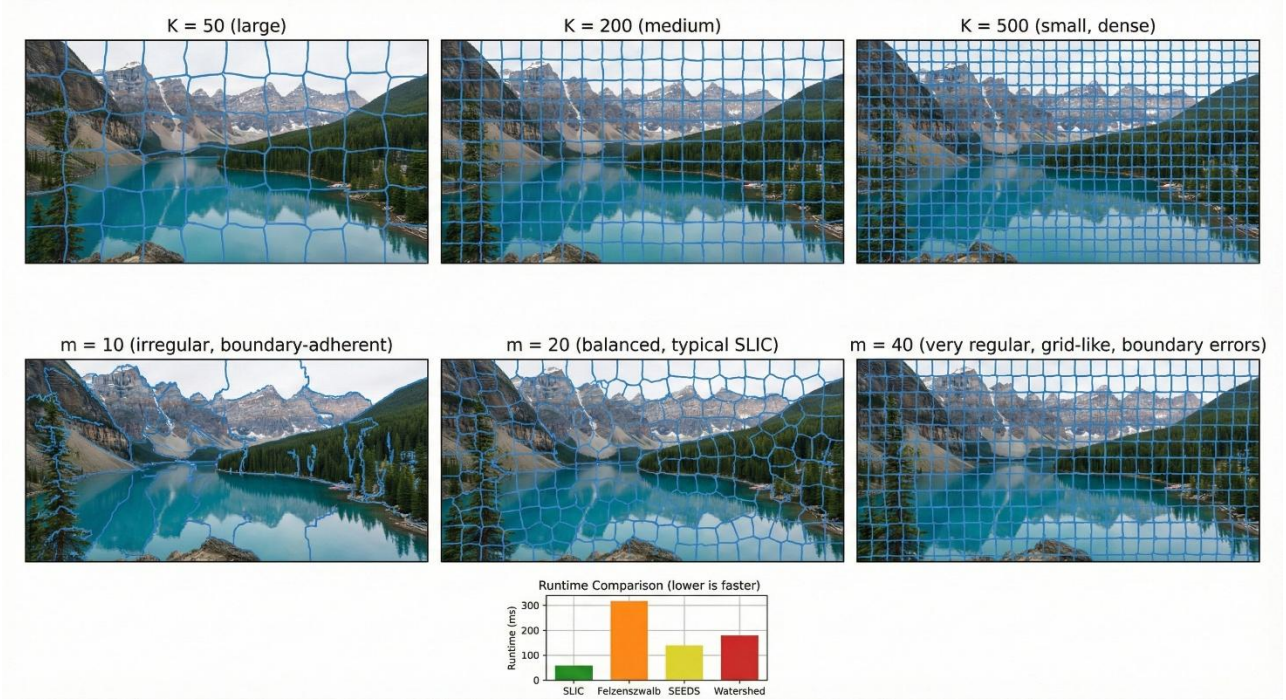
# SLIC Superpixels — Simple & Fast Clustering

## Why It's the Standard Choice

- Extremely fast (roughly 10× faster than many alternatives)
- Simple idea: K-Means applied to color + spatial position
- Memory-light and stable
- Produces superpixels that follow object boundaries well

## How It Works (Conceptually)

- Start with a grid of initial cluster centers.
- For each pixel, search only in a small local neighborhood (not the whole image).
- Group each pixel with the nearest center based on color + location similarity.
- Update the centers and repeat until stable.
- Fix tiny stray regions to ensure connectivity.




### What You Control

- **K (number of superpixels):** fewer = larger regions, more = finer details
- **m (compactness):**
  - Low → irregular shapes, excellent boundary accuracy
  - High → more regular shapes, but worse boundary adherence

# Thank You

For Your Attention